

AINN Practical Description

Practical No 01

1. Given a 2D grid maze, where some cells are blocked (obstacles) and others are free paths, implement a pathfinding AI that finds the shortest route from a start position (S) to a goal position (G) using:
a. 1. A* Algorithm b. 2. Greedy Best-First Search (GBFS)

Practical: Pathfinding AI using A* and GBFS

Aim

To find a path from Start (S) to Goal (G) in a 2D maze using:

- A* Algorithm
 - Greedy Best-First Search (GBFS)
-

Theory

Greedy Best-First Search (GBFS)

- Uses only the heuristic value $h(n)$.
- Chooses the node closest to the goal.
- Fast but may not always find the shortest path.

Heuristic (Manhattan Distance):

$$h(n) = |x_1 - x_2| + |y_1 - y_2| \quad h(n) = |x_1 - x_2| + |y_1 - y_2| \quad h(n) = |x_1 - x_2| + |y_1 - y_2|$$

A* Algorithm

- Uses actual cost $g(n)$ and heuristic cost $h(n)$.
- Finds the shortest and optimal path.

Formula:

$$f(n) = g(n) + h(n) \quad f(n) = g(n) + h(n) \quad f(n) = g(n) + h(n)$$

where:

- $g(n)$ = cost from start node
 - $h(n)$ = estimated cost to goal
 - $f(n)$ = total cost
-

Algorithm

GBFS

1. Add start node to priority queue.
2. Select node with smallest heuristic value.
3. Visit neighboring nodes.
4. Repeat until goal is reached.
5. Return the path.

A*

1. Add start node to priority queue.
2. Calculate $f(n) = g(n) + h(n)$.
3. Select node with smallest $f(n)$.
4. Update costs and parent nodes.
5. Repeat until goal is reached.
6. Return the shortest path.

Practical No 02

Develop a Python program to solve a CSP-based Sudoku Solver and a Map Coloring Problem using Backtracking Search with heuristics.

Practical: CSP-Based Sudoku Solver using Backtracking

Purpose of the Practical

The purpose of this practical is to learn how **Constraint Satisfaction Problems (CSPs)** can be solved using **Backtracking Search**. It helps in solving a Sudoku puzzle by satisfying all constraints of rows, columns, and 3×3 sub-grids.

Aim

To solve a Sudoku puzzle using **Constraint Satisfaction Problem (CSP)** and **Backtracking Search**.

Theory

- Sudoku is a CSP problem.
- Numbers 1–9 must appear only once in each row, column, and 3×3 grid.
- Backtracking tries numbers and removes incorrect choices.

Algorithm

1. Find an empty cell.
2. Try numbers from 1 to 9.
3. Check row, column, and sub-grid constraints.
4. If valid, place the number.
5. Recursively solve remaining cells.
6. If no number fits, backtrack.

7. Continue until the puzzle is solved.

Functions Used

is_safe()

- Checks whether a number can be placed in a cell.

solve_sudoku()

- Solves the Sudoku puzzle using recursive Backtracking.

Practical No 03

Develop a knowledge base for a medical diagnosis system using First Order Logic predicates such as: a. HasSymptom(John, Flu) b. HasDisease(Kento, diarrhea)

Purpose

To understand how **First Order Logic (FOL)** can be used to represent medical knowledge and infer diseases based on symptoms.

Aim

To develop a **Medical Diagnosis System** using **First Order Logic predicates** and inference rules.

Theory

- First Order Logic (FOL) is used to represent facts and rules.
- Facts store known information about symptoms.
- Rules are used to infer diseases from symptoms.
- The system applies logical reasoning to reach conclusions.

Predicates Used

- **HasSymptom(John, Fever)**
- **HasSymptom(John, Cough)**
- **HasDisease(John, Flu)**
- **NeedsRest(John)**

Algorithm

1. Store symptoms as facts.
2. Define diagnosis rules.
3. Check if rule conditions are satisfied.
4. Infer disease or treatment recommendation.

5. Display the final diagnosis.

Functions Used

facts

- Stores known symptoms and diseases.

rules

- Stores logical inference rules.

Inference Process

- Applies rules repeatedly until no new facts are generated.

Practical No 04

4. Develop a medical diagnosis system using First-Order Logic (FOL) that can infer diseases based on symptoms and conditions. Given Knowledge Base (KB) in FOL: a. $\forall x (\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x))$ b. $\forall x (\text{Flu}(x) \rightarrow \text{NeedsRest}(x))$ c. $\forall x (\text{SevereCough}(x) \wedge \text{ShortnessOfBreath}(x) \rightarrow \text{Pneumonia}(x))$ d. $\text{Fever}(\text{John})$ e. $\text{Cough}(\text{John})$

Purpose

To learn how **First-Order Logic (FOL)** and **Forward Chaining** can be used to infer diseases from symptoms automatically.

Aim

To develop a medical diagnosis system using **First-Order Logic (FOL)** that can infer diseases based on symptoms and conditions.

Theory

- First-Order Logic (FOL) represents knowledge using facts and rules.
- Facts store known symptoms of a patient.
- Rules are used to infer diseases from symptoms.
- Forward Chaining applies rules repeatedly to generate new facts.

Knowledge Base

1. $\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$
2. $\text{Flu}(x) \rightarrow \text{NeedsRest}(x)$
3. $\text{SevereCough}(x) \wedge \text{ShortnessOfBreath}(x) \rightarrow \text{Pneumonia}(x)$

4. Fever(John)

5. Cough(John)

Algorithm

1. Store symptoms as facts.
2. Define medical rules.
3. Apply Forward Chaining.
4. Generate new facts when rule conditions are satisfied.
5. Continue until no new facts can be inferred.
6. Display diagnosis results.

Functions Used

substitute()

- Replaces variable **x** with a person's name.

forward_chain()

- Applies rules repeatedly to infer new facts.

explain()

- Shows how a conclusion was derived.

Practical No 05

Implement Linear Regression using Batch Gradient Descent (BGD), Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent (MBGD). a. Given a dataset of house prices (features: size, number of rooms), use Gradient Descent to optimize model parameters. b. Compare the convergence rates of BGD, SGD, and MBGD.

Purpose

To understand how different **Gradient Descent algorithms** optimize the parameters of a Linear Regression model and compare their convergence rates.

Aim

To implement **Linear Regression** using:

- Batch Gradient Descent (BGD)
- Stochastic Gradient Descent (SGD)
- Mini-Batch Gradient Descent (MBGD)

and compare their performance on a house price dataset.

Theory

- Linear Regression predicts house prices using features such as **size** and **number of rooms**.
- Gradient Descent minimizes the error by updating model parameters.

1. Batch Gradient Descent (BGD)

- Uses the entire dataset for each update.
- Stable but slower for large datasets.

2. Stochastic Gradient Descent (SGD)

- Uses one training example at a time.
- Faster but updates are noisy.

3. Mini-Batch Gradient Descent (MBGD)

- Uses a small batch of data for each update.
- Faster than BGD and more stable than SGD.

Algorithm

1. Initialize model parameters.
2. Calculate prediction error.
3. Compute gradients.
4. Update parameters using Gradient Descent.
5. Repeat until convergence.
6. Compare convergence of BGD, SGD, and MBGD.

Functions Used

Linear Regression Model

- Predicts house prices.

Gradient Descent

- Updates model parameters to reduce error.

BGD

- Uses complete dataset.

SGD

- Uses one sample at a time.

MBGD

- Uses a small batch of samples.

Practical No 06

Handwritten Digit Recognition (MNIST Dataset)-Implement a 3-layer neural network (Input → Hidden → Output) using Backpropagation. a. Use Sigmoid, ReLU, or Tanh activation functions. b. Train the network on the MNIST dataset to classify handwritten digits (0-9). c. Optimize using Gradient Descent with Momentum or Adam Optimizer.

Purpose

To learn how a **Neural Network** uses **Backpropagation** to recognize and classify handwritten digits from the MNIST dataset.

Aim

To implement a **3-Layer Neural Network (Input → Hidden → Output)** using Backpropagation for handwritten digit recognition.

Theory

- MNIST is a dataset containing handwritten digit images (0–9).
- A Neural Network learns patterns from these images and predicts the correct digit.
- Backpropagation adjusts weights to reduce prediction errors.
- Activation functions such as **Sigmoid, ReLU, or Tanh** introduce non-linearity.
- Optimizers like **Gradient Descent with Momentum** or **Adam** improve training efficiency.

Network Architecture

1. **Input Layer** – Receives pixel values of digit images.
2. **Hidden Layer** – Extracts important features.
3. **Output Layer** – Predicts digits from 0 to 9.

Algorithm

1. Load the MNIST dataset.
2. Normalize image pixel values.
3. Create a 3-layer neural network.
4. Apply activation functions (ReLU/Sigmoid/Tanh).
5. Perform forward propagation.
6. Calculate prediction error.
7. Use backpropagation to update weights.
8. Optimize using Adam or Momentum.
9. Repeat training for multiple epochs.
10. Classify handwritten digits.

Functions Used

Activation Function

- ReLU, Sigmoid, or Tanh for neuron activation.

Forward Propagation

- Computes network output.

Backpropagation

- Updates weights using error gradients.

Adam Optimizer

- Optimizes learning and speeds up convergence.

Practical No 07

7. Clustering 2D Data Using ART1- a. Implement the ART1 algorithm for binary input data. b. Given a set of binary vectors, cluster them into categories based on similarity. c. Use vigilance parameter (ρ) to control cluster formation.

Purpose

To group similar binary data (0s and 1s) into clusters using the ART1 algorithm.

Aim

To implement the ART1 algorithm and create clusters based on similarity between binary vectors.

Theory

- ART1 is a clustering algorithm.
- It works with binary data (0 and 1).
- Similar data are placed in the same cluster.
- The vigilance parameter (ρ) decides how similar the data must be to join a cluster.
 - High $\rho \rightarrow$ More clusters.
 - Low $\rho \rightarrow$ Fewer clusters.

Algorithm

1. Take binary input vectors.
2. Compare them with existing clusters.
3. If similarity is high, add to the same cluster.

4. Otherwise, create a new cluster.
5. Repeat for all vectors.

Input

V1 = [1 1 0 0]

V2 = [1 1 0 1]

V3 = [0 0 1 1]

V4 = [0 1 1 1]

Output

Cluster 1: V1, V2

Cluster 2: V3, V4

Result

Successfully implemented the **ART1 clustering algorithm** and grouped similar binary vectors into different clusters using the vigilance parameter (ρ).

Easy one-line purpose:

To group similar binary data into clusters using the ART1 algorithm.

Practical No 09

Object Recognition using CNN (CIFAR-10 Dataset) a. Train a CNN model to classify CIFAR-10 images into 10 object categories (airplane, cat, car, dog, etc.). b. Use data augmentation to improve model generalization. c. Compare model performance with and without Batch Normalization and Dropout.

Purpose

To learn how a **Convolutional Neural Network (CNN)** can recognize and classify objects in images.

Aim

To train a **CNN model** on the **CIFAR-10 dataset** to classify images into 10 object categories.

Theory

- CIFAR-10 contains 60,000 color images belonging to 10 classes.
- CNN automatically extracts image features and classifies objects.
- Data Augmentation creates modified images to improve accuracy and generalization.
- Batch Normalization speeds up training and improves stability.
- Dropout reduces overfitting by randomly deactivating neurons during training.

Object Categories

- Airplane
- Automobile (Car)
- Bird

- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

Algorithm

1. Load the CIFAR-10 dataset.
2. Preprocess and normalize images.
3. Apply data augmentation.
4. Build the CNN model.
5. Train the model.
6. Evaluate model accuracy.
7. Compare results with and without Batch Normalization and Dropout.

Functions Used

Convolution Layer

- Extracts image features.

Pooling Layer

- Reduces image dimensions.

Batch Normalization

- Improves training stability.

Dropout

- Prevents overfitting.

Softmax

- Predicts object class.

Output

Predicted Class : Airplane

Actual Class : Airplane

Model Accuracy:

Without BN & Dropout : 75%

With BN & Dropout : 85%

Practical No 10

10. Implementing Self-Organizing Maps (SOM) for Image Compression- a. Implement Kohonen's Self-Organizing Map (SOM) for image compression. b. Train the SOM to reduce the number of colors in an image. c. Display the original and compressed images to analyze quality loss.

To learn how **Self-Organizing Maps (SOM)** can compress an image by reducing the number of colors while preserving image quality.

Aim

To implement **Kohonen's Self-Organizing Map (SOM)** for image compression and compare the original and compressed images.

Theory

- SOM is an unsupervised neural network.
- It groups similar colors together.
- The number of colors in the image is reduced.
- This decreases image size while maintaining visual quality.
- Some quality loss may occur after compression.

Algorithm

1. Load the image.
2. Extract pixel color values.
3. Initialize the SOM network.
4. Train SOM using image pixels.
5. Replace similar colors with representative colors.
6. Generate the compressed image.

7. Compare original and compressed images.

Functions Used

SOM Training

- Learns and groups similar colors.

Color Reduction

- Reduces the number of colors in the image.

Image Reconstruction

- Creates the compressed image using reduced colors.

Output

Original Image Size : 500 KB

Compressed Image Size : 200 KB

Colors Before Compression : 16,777,216

Colors After Compression : 256